Aplicación web en React conectada a una API REST

PowerFit. Programación Web

Javier Carrera

1. Introducción

En esta actividad se desarrolló una aplicación web con React para consultar y administrar la información del gimnasio PowerFit. Los datos no están escritos en los componentes: se obtienen mediante solicitudes HTTP a la API REST hecha con Spring Boot y Java en el bloque anterior.

2. Revisión de la API REST

La API responde en http://localhost:8080/api con formato JSON. Casi todos sus servicios requieren un token JWT, que se obtiene con POST /auth/login, y algunos dependen del rol del usuario. Los recursos que usa la aplicación son:

- **Planes** (/planes): GET para consultar y PUT para cambiar precio y descripción.
- **Socios** (/socios): GET, PUT y DELETE. Los socios se crean con el registro y la aprobación de cuentas, no con un POST en este recurso.
- **Entrenadores** (/entrenadores): GET, PUT y DELETE.
- **Clases** (/clases): GET, POST, PUT y DELETE.
- **Membresías** (/membresias): GET, POST, PUT y DELETE, además de PATCH para cancelar. El socio usa /mias y /contratar.
- **Pagos** (/pagos): GET, POST, PUT y DELETE. El socio usa /mios y /pagar.
- **Inscripciones** (/inscripciones): GET, POST y DELETE. El socio usa /mias.

Por ejemplo, GET /planes devuelve una lista de objetos como este:

```
{ "id": 2, "tipo": "MENSUAL", "nombre": "Plan Mensual",
  "descripcion": "Acceso libre por 30 días...", "precio": 30.00, "duracionDias": 30 }
```

Cuando hay un error, la API devuelve el código HTTP, un mensaje y, si falla la validación, un objeto errores con el problema de cada campo. La aplicación usa esos datos para informar al usuario.

3. Creación de la aplicación

El proyecto está en la carpeta react-app. Se creó con React y Vite, y se usó React Router para la navegación entre páginas. Las solicitudes se hacen con fetch, sin librerías adicionales. El código está organizado así:

- src/api: el cliente HTTP y una función por cada endpoint.
- src/context: la sesión del usuario y los avisos.
- src/hooks: useConsulta, que controla los estados de cada consulta.
- src/components: las partes reutilizables de la interfaz.
- src/pages: una página por cada recurso.

4. Comunicación entre React y la API

La dirección de la API se configura en el archivo .env, por lo que no está repetida en el código:

```
VITE_API_URL=http://localhost:8080/api
```

La aplicación se ejecuta en el puerto 5173, que la API ya permitía en su configuración de CORS (app.cors.origenes). Sin ese permiso, el navegador bloquearía las respuestas.

Todas las solicitudes pasan por la función peticion de src/api/cliente.js, que agrega el token y convierte las respuestas de error en mensajes:

```
export async function peticion(ruta, { metodo = 'GET', datos } = {}) {
  const token = obtenerToken();
  const headers = {};
  if (datos !== undefined) headers['Content-Type'] = 'application/json';
  if (token) headers.Authorization = `Bearer ${token}`;
  // ...hace el fetch, revisa la respuesta y devuelve el JSON
}
```

El proceso completo funciona de la siguiente manera:

1. El usuario inicia sesión y React envía sus datos a POST /auth/login.
2. La API devuelve el token y los datos del usuario, que se guardan en localStorage.
3. Al abrir una página, useConsulta llama al servicio correspondiente, por ejemplo GET /clases.
4. La API responde con JSON, React actualiza el estado y los componentes muestran los datos.
5. Al guardar o eliminar, se envía la solicitud y, si es correcta, la lista se vuelve a consultar.

5. Información y operaciones

Los socios, membresías, pagos e inscripciones se muestran en tablas, mientras que las clases, entrenadores y planes aparecen en tarjetas. La página de inicio calcula un resumen con los datos de la API, como socios registrados y saldo pendiente. Los socios se pueden buscar y las clases se pueden filtrar por día.

La aplicación permite consultar, registrar, editar y eliminar según lo que ofrece cada recurso. Antes de eliminar pide confirmación. El menú y los botones cambian según el rol: por ejemplo, un entrenador solo edita sus propias clases y un socio puede contratar un plan, pagar e inscribirse.

No se incluyeron el registro público de cuentas ni la aprobación de usuarios (/auth/registro y /admin/usuarios). Esas funciones siguen disponibles en la aplicación Angular.

6. Estados y errores

El componente EstadoConsulta muestra un estado distinto según el resultado de cada consulta:

- **Cargando:** un indicador con el texto “Cargando información…”.
- **Información disponible:** la tabla o las tarjetas con los datos.
- **Sin registros:** un mensaje como “No existen membresías registradas”.
- **Error al consultar:** el motivo y un botón para reintentar. Si la API está apagada, indica que se compruebe que el servidor esté encendido.

Al registrar o modificar, los campos obligatorios se revisan antes de enviar. Si la API rechaza los datos, el mensaje aparece en el propio formulario, por ejemplo “La hora de fin debe ser posterior a la hora de inicio”, y los errores de validación se muestran junto a cada campo. Si la sesión vence, la aplicación vuelve al inicio de sesión y lo avisa.

7. Componentes

La interfaz se divide en estos componentes principales:

- **Principal y navegación:** App define las rutas, Layout la estructura común, Menu las opciones y RutaProtegida controla el acceso.
- **Listas y tarjetas:** TablaDatos sirve para cualquier recurso, y TarjetaClase, TarjetaEntrenador y TarjetaPlan muestran un registro cada una.
- **Formularios:** ModalFormulario se reutiliza en todos los recursos. Cada página solo le indica sus campos, y Campo dibuja cada uno con su etiqueta y error.
- **Mensajes:** Estados para las consultas, ConfirmarAccion para confirmar y AvisosContext para los mensajes emergentes.

Gracias a esta separación, por ejemplo, las siete páginas que tienen formulario o confirmación usan los mismos componentes.

8. Pruebas

Para no modificar los datos reales ni enviar correos, las operaciones de escritura se probaron con una base de datos temporal, datos ficticios y el correo desactivado. Se usó un navegador automatizado con la API real y se comprobó lo siguiente:

- La conexión de React con la API y que los datos se muestran correctamente.
- Registrar, editar y eliminar en los recursos disponibles, incluidas las acciones que la API rechaza, como eliminar una clase con socios inscritos.
- Los cuatro roles: administrador, recepcionista, entrenador y socio.
- Los estados de carga, sin registros, sesión vencida y API apagada.
- Que a 375 px de ancho las páginas no tienen desplazamiento lateral.

También se recorrieron las pantallas con la base real, solo en modo de lectura, y se mostraron sin errores. Las pruebas ayudaron a corregir varios fallos, como avisos que se acumulaban sobre los botones y errores de formulario que no recibían el foco. No se probó en un teléfono físico ni con un lector de pantalla.

9. Evidencias

Las capturas usan datos ficticios. El conjunto completo está en react-app/evidencias.

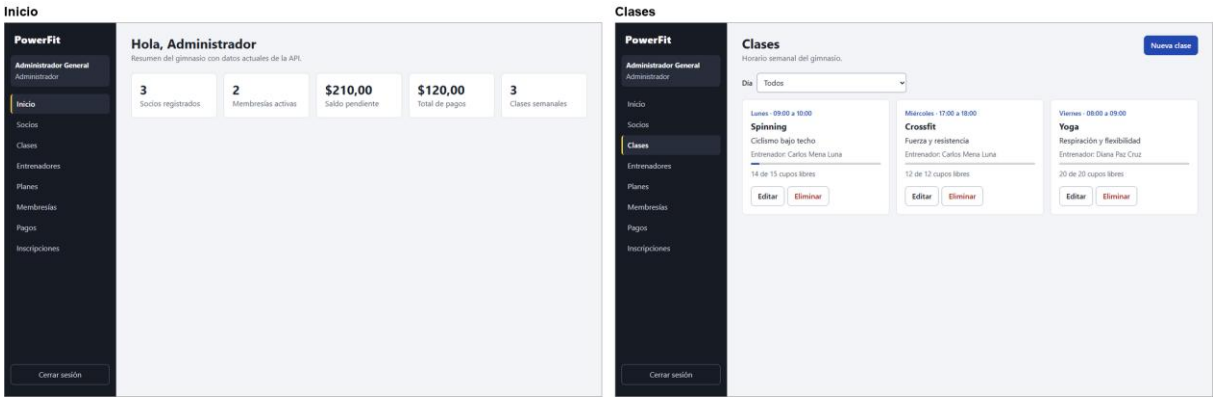


Figura 1. Inicio con datos de la API y clases en tarjetas.

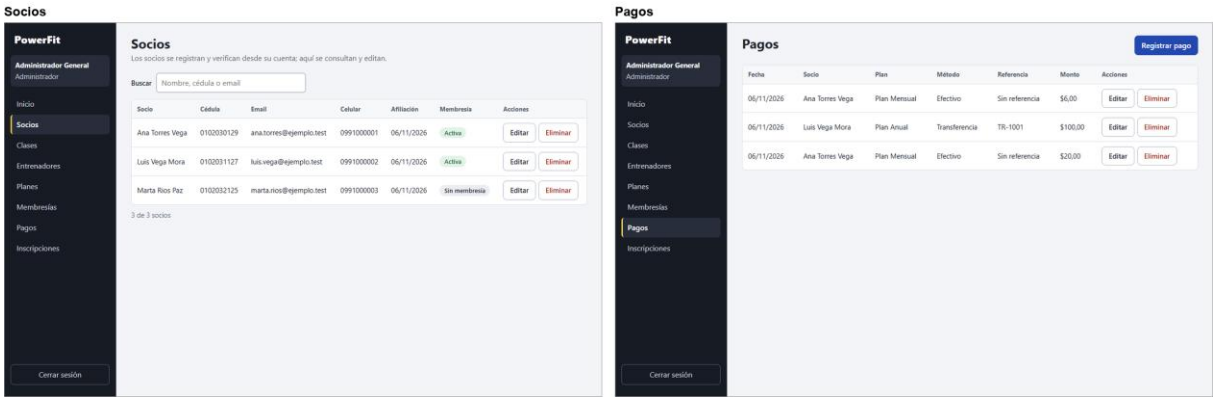


Figura 2. Socios y pagos en tablas.

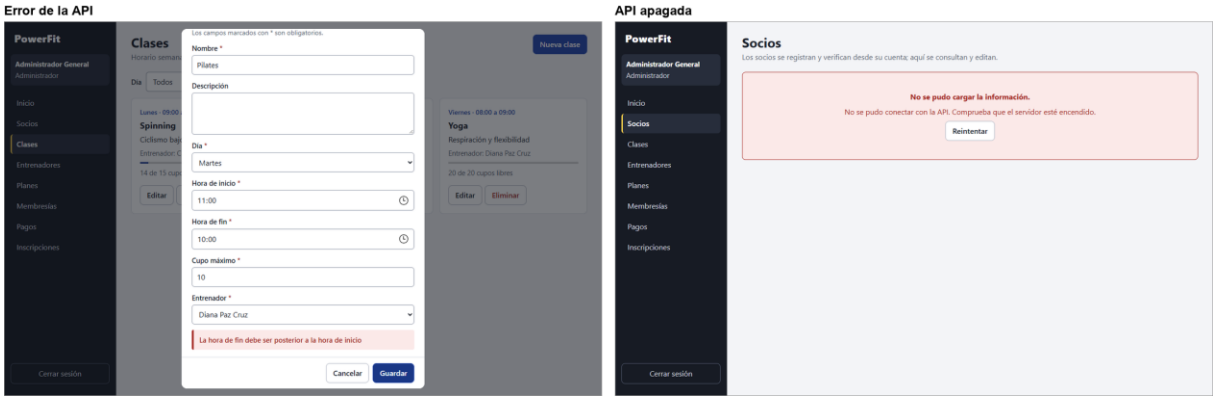


Figura 3. Error de la API en un formulario y aplicación con la API apagada.

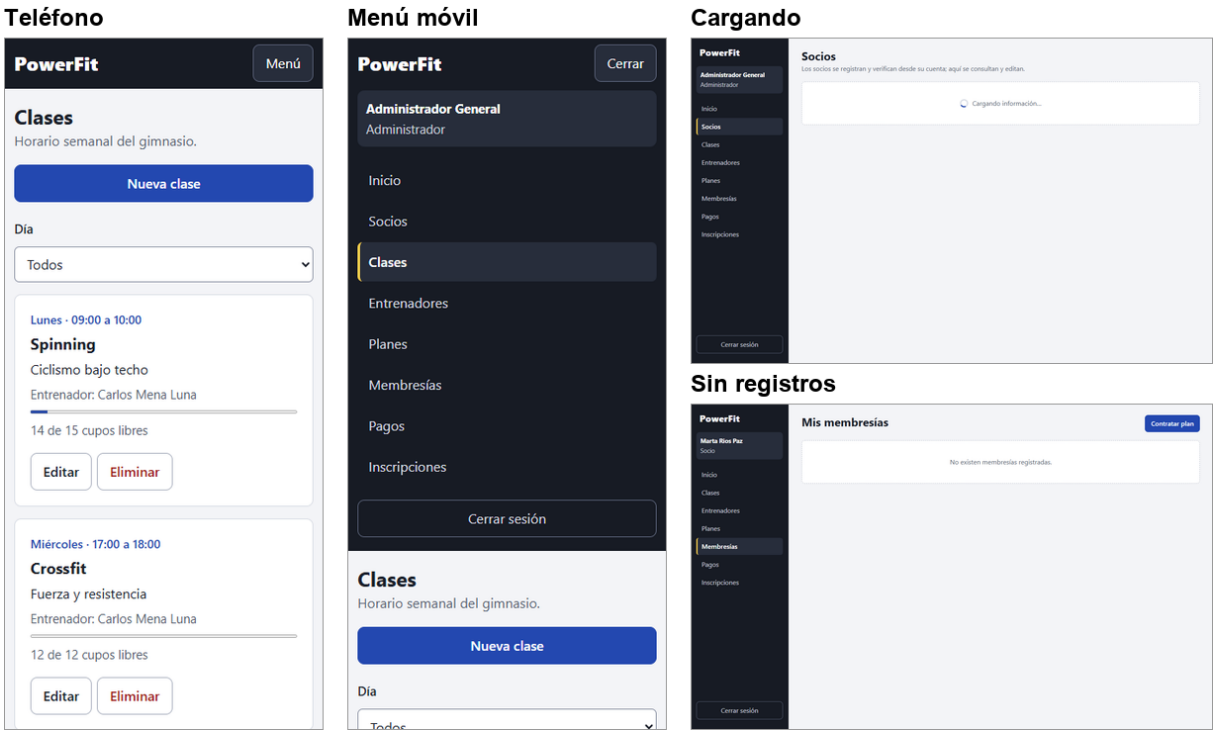


Figura 4. Vista en teléfono, menú móvil, estado de carga y lista sin registros.

10. Ejecución

Primero se inicia MySQL y la API de Spring Boot. Después, dentro de react-app, se ejecuta npm install y npm run dev, y se abre http://localhost:5173. Si la API cambia de dirección, basta con modificar VITE_API_URL en el archivo .env y reiniciar la aplicación.

11. Conclusiones

La actividad permitió ver cómo el frontend y el backend trabajan por separado y se comunican solo mediante HTTP y JSON. Tener un único cliente HTTP facilitó manejar el token y los errores en un solo lugar, y los componentes reutilizables evitaron repetir formularios y tablas.